

bob131_container_death_triage

bob131_container_death_triage.sh

Revision: 1 **Last modified:** 2026-08-21T17:06:26Z **Script:** [scripts/diagnostics/bob131_container_death_triage.sh](#) **Companion doc per §11.4.18.** Update both in the same commit.

Overview

Classifies why a rootless-podman container stopped, into one of six mutually exclusive classes, from evidence rather than from a guess. Written for BOB-131, where a single ticket recorded “podman common crash” for two unrelated events — a host power-off and a SIGSEGV inside the containerised process — and neither was a common crash.

The reasoning it encodes is documented in [docs/guides/container-death-triage.md](#).

Prerequisites

bash, awk, grep. collect additionally needs podman and journalctl and read access to /sys/fs/cgroup. classify and --selftest need neither podman nor the journal, so they run anywhere.

Usage

```
S=./scripts/diagnostics/bob131_container_death_triage.sh
```

```
$S --selftest fixtures # 8 golden
```

```
$S collect <container> <since> <until> > bundle.txt
      # read-only evidence bundle
$S classify bundle.txt                                # VERDICT
      + NOTE lines
```

<since> / <until> are host-local journalctl timestamps ("2026-08-20 17:55:30").

Example

```
$ $S collect qbittorrent-proxy "2026-08-20 17:55:30" "2026-08-20
17:57:00" > /tmp/b.txt
$ $S classify /tmp/b.txt
VERDICT: PROCESS-SIGSEGV
NOTE: exit 139 (=128+11) and/or a kernel SIGSEGV/ANOM_ABEND
record:
NOTE: the container's PID 1 died of SIGSEGV.
...
```

Classes

HOST-POWER-TRANSITION · THREAD-LIMIT-EXHAUSTION · CGROUP-OOM-KILL ·
PROCESS-SIGSEGV · ORCHESTRATED-STOP · CGROUP-MEMORY-CEILING · UNKNOWN

Evaluated in that order, most-specific first. UNKNOWN is a real answer: it means no class matched, not that the tool failed.

Internal behaviour

collect writes a plain-text bundle of ### SECTION blocks (EXIT_CODE, EVENTS, KERNEL, LOGIND, LOGS, COMMON, CGROUP, LIMITS). classify reads only that bundle, so verdicts are reproducible offline and fixtures are just bundles.

Two deliberate source choices, both §11.4.201:

- **EXIT_CODE comes from the died exit= event inside the window**, falling back to podman inspect only when the window has none. Reading it from inspect alone would describe whatever container carries that *name* today — a different container for any historical window.
- **CGROUP is read live** and the bundle says so. A dead container's cgroup no longer exists, so these numbers describe present pressure, never the incident's.

Side effects

None. Every podman, journal and cgroup access is read-only. The script starts, stops, restarts, removes and signals nothing — it issues no signal at all, so it cannot reach $\text{pgid} \leq 1$ (§11.4.263, Hard Stop #3).

Edge cases

- `podman events` can return nothing for a window that demonstrably contains events. Verdicts do not depend on it; the journal sections carry the weight.
- A bundle with no evidence at all classifies `UNKNOWN`, which is also the negative control's expected result.
- `CGROUP-MEMORY-CEILING` is emitted as a primary verdict only when nothing else matched. Otherwise it appears as a `NOTE`: — pressure, not cause.

Testing

```
$S --selftest      # exit 0 on 8/8
```

Eight fixtures: one per class, a **negative control** (quiet container $\Rightarrow$ `UNKNOWN`) guarding against false positives, and a **common-carrier** fixture asserting that a common `<error>`: line does not divert the verdict.

Verified 2026-08-21: 5 consecutive runs all PASS (8/8), `rc=0` (§11.4.50).
Paired §1.1 mutations both make it FAIL:

Mutation	Result
neuter the power-off detector	FAIL (1/8 wrong)
collapse <code>oom_kill > 0</code> into <code>oom_kill >= 0</code>	FAIL (5/8 wrong) — including the negative control

Related

- [docs/guides/container-death-triage.md](#) — the triage reasoning
- [docs/incidents/2026-08-21-bob131-container-death-triage.md](#) — the BOB-131 investigation
- [scripts/flight-recorder/](#) — continuous session-health sampling (OOM cgroup attribution, thread ramp, PSI)

Last verified: 2026-08-21